WNDRVR
SUPPORT NETWORK

VXWORKS 7 BOOTING THE VXWORKS KERNEL WITH THE U-BOOT BOOT LOADER TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Booting the VxWorks Kernel with the U-Boot Boot Loader Tutorial, SR0650

2 July 2020

1. ABOUT THE U-BOOT BOOT LOADER TUTORIAL

For ARM, PowerPC, and RISC-V targets, you can configure, build, and use the U-Boot boot loader to boot a VxWorks® 7 kernel.

Objective

This tutorial illustrates how to obtain, install, and build a U-Boot boot loader for a PowerPC P2020rdb target. It describes how to flash the boot loader on the target and use it to boot a fully-featured VxWorks 7 kernel with a separate device tree binary (DTB) from flash.

Note: Completing this tutorial takes approximately 90 minutes.

Prerequisites

This use case requires the following:

- An ARM or PowerPC target supported by your VxWorks 7 installation. (This use case is not applicable to Intel Architecture targets.)
- A VxWorks image for your target that includes support for U-Boot.
- A suitable toolchain (such as the GNU compiler) for building U-Boot. For more information, see Installing and Configuring a Toolchain to Build U-Boot on page 5.

For more information on the available boot loaders for VxWorks 7 (including U-Boot), see the VxWorks 7 Boot Loader User's Guide.

Overview

You will perform the following tasks:

- Obtain and install a U-Boot source tree on your host.
- Configure U-Boot and build it for your target.
- Use U-Boot to boot a fully-featured VxWorks 7 kernel.
- Use U-Boot scripting to make work more efficient.

Audience

This information is for engineers and programmers that are tasked with installing and using U-Boot on a target board to boot VxWorks 7.

2. BOOTING THE VXWORKS KERNEL WITH U-BOOT

1. Obtaining and Installing the U-Boot Source Tree

To create a U-Boot boot loader for VxWorks, you must first download and install the U-Boot source tree.

Procedure

1. Obtain the U-Boot installation from Denx as a tarball.

   The download is available from the following website:

   ftp://ftp.denx.de/pub/u-boot/

   > Note: Alternatively, you could obtain the install from the git repository at: http://git.denx.de

2. Untar the tarball to your install directory at **/home/wruser/u-boot**.

   > Note: Regardless of which method you choose (download or git), there is no installation process. The process only requires that you bring the source tree into a directory on your development host.

Results

Your U-Boot installation directory is as follows:

```
[wruser@wrhost u-boot]$ ls
api          config.mk  dts       Licenses  net       snapshot.commit
arch         CREDITS    examples  MAKEALL   post      spl
board        disk       fs        Makefile  README    System.map
boards.cfg   doc        include   mkconfig  rules.mk  test
common       drivers    lib       nand_spl  scripts   tools
```

Next topic: Updating the U-Boot Configuration Files on page 2

2. Updating the U-Boot Configuration Files

In some cases, you must modify the U-Boot configuration files to support your target hardware.

About This Task

The standard U-Boot installation includes support for hundreds of device drivers to support buses, peripherals, storage devices, and so forth. This makes it a suitable starting point for most systems, regardless of your configuration. In most cases, even if a device driver for your specific device is not present, you can use a similar device as a starting point.

You can create or modify the main configuration file for your target hardware or custom board as needed. The configuration file defines hardware parameters such as address spaces, software features such as **ping**, supported devices (Ethernet controllers, serial, and so forth), and default values for environment variables.

If the installation already includes a configuration file that matches your target hardware, Wind River recommends that you initially leave the file with its default values.

If there is no configuration file for your target board, start with a configuration file for a board that is similar in hardware configuration to your board. In this case, you must rename the file and modify it to match your hardware as needed.

Procedure

1. Confirm that there is an existing board configuration file for your hardware and that it suits your needs.

   Board configuration files are located under the following directory:

   /home/wruser/u-boot/include/configs

   Confirm that the **P1_P2_RDB.h** file is available. This file supports the P2020rdb board.

2. Open the **P1_P2_RDB.h** file in an editor.

   The following example shows some configuration options available in this file:

```
...
#ifndef CONFIG_SYS_TEXT_BASE
#define CONFIG_SYS_TEXT_BASE            0xeff80000
#endif
...
#define CONFIG_PCI              1       /* Enable PCI/PCIE */
#if defined(CONFIG_PCI)
#define CONFIG_PCIE1            1       /* PCIE controler 1 (slot 1) */
#define CONFIG_PCIE2            1       /* PCIE controler 2 (slot 2) */
#define CONFIG_FSL_PCI_INIT     1       /* Use common FSL init code */
#define CONFIG_PCI_INDIRECT_BRIDGE 1    /* indirect PCI bridge support */
#define CONFIG_FSL_PCIE_RESET   1       /* need PCIe reset errata */
#define CONFIG_SYS_PCI_64BIT    1       /* enable 64-bit PCI resources */
#endif /* #if defined(CONFIG_PCI) */
...
#define CONFIG_DDR_CLK_FREQ     66666666 /* DDRCLK on P1_P2 RDB */
#define CONFIG_SYS_CLK_FREQ     get_board_sys_clk(0) /*sysclk for P1_P2 RDB */
...
#define CONFIG_L2_CACHE                 /* toggle L2 cache */
...
#define CONFIG_SYS_MEMTEST_START        0x00000000      /* memtest works on */
#define CONFIG_SYS_MEMTEST_END          0x1fffffff
...
#define CONFIG_SYS_CCSRBAR              0xffe00000
#define CONFIG_SYS_CCSRBAR_PHYS_LOW     CONFIG_SYS_CCSRBAR
...
#define CONFIG_SYS_SDRAM_SIZE   1024    /* DDR size on P1_P2 RDBs */
#define CONFIG_SYS_DDR_SDRAM_BASE       0x00000000
#define CONFIG_SYS_SDRAM_BASE           CONFIG_SYS_DDR_SDRAM_BASE

#define CONFIG_NUM_DDR_CONTROLLERS      1
#define CONFIG_DIMM_SLOTS_PER_CTLR      1
#define CONFIG_CHIP_SELECTS_PER_CTRL    1
```

```
#define CONFIG_SYS_DDR_ERR_INT_EN       0x0000000d
#define CONFIG_SYS_DDR_ERR_DIS          0x00000000
#define CONFIG_SYS_DDR_SBE              0x00FF0000
...
/* Serial Port - controlled on board with jumper J8
 * open - index 2
 * shorted - index 1
 */
#define CONFIG_CONS_INDEX       1
#define CONFIG_SYS_NS16550
#define CONFIG_SYS_NS16550_SERIAL
#define CONFIG_SYS_NS16550_REG_SIZE     1
#define CONFIG_SYS_NS16550_CLK          get_bus_freq(0)
...
#define CONFIG_SYS_NS16550_COM1 (CONFIG_SYS_CCSRBAR+0x4500)
#define CONFIG_SYS_NS16550_COM2 (CONFIG_SYS_CCSRBAR+0x4600)
...
#if defined(CONFIG_TSEC_ENET)
#define CONFIG_MII              1       /* MII PHY management */
#define CONFIG_MII_DEFAULT_TSEC 1       /* Allow unregistered phys */
#define CONFIG_TSEC1    1
#define CONFIG_TSEC1_NAME       "eTSEC1"
#define CONFIG_TSEC2    1
#define CONFIG_TSEC2_NAME       "eTSEC2"
#define CONFIG_TSEC3    1
#define CONFIG_TSEC3_NAME       "eTSEC3"

#define TSEC1_PHY_ADDR          2
#define TSEC2_PHY_ADDR          0
#define TSEC3_PHY_ADDR          1
...
/*
 * Command line configuration.
 */
#include <config_cmd_default.h>

#define CONFIG_CMD_DATE
#define CONFIG_CMD_ELF
#define CONFIG_CMD_I2C
#define CONFIG_CMD_IRQ
#define CONFIG_CMD_MII
#define CONFIG_CMD_PING
#define CONFIG_CMD_SETEXPR
#define CONFIG_CMD_REGINFO
...
#define CONFIG_NFSBOOTCOMMAND           \
        "setenv bootargs root=/dev/nfs rw "     \
        "nfsroot=$serverip:$rootpath "          \
        "ip=$ipaddr:$serverip:$gatewayip:$netmask:$hostname:$netdev:off " \
        "console=$consoledev,$baudrate $othbootargs;"   \
        "tftp $loadaddr $bootfile;"             \
        "tftp $fdtaddr $fdtfile;"               \
        "bootm $loadaddr - $fdtaddr"
```

3. Confirm that the information for the P2020rdb board is included in the following file: **/home/wruser/u-boot/boards.cfg**.

   For the P2020rdb board, the line is as follows:

```
Active powerpc mpc85xx - freescale p1_p2_rdb P2020RDB P1_P2_RDB:P2020RDB Poonam Aggrwal
```

Note: The general syntax for adding a new board is as follows:

status architecture cpuCore soc vendorName boardName targetName options ownerName

Where:

status

Can be set to Active or Orphan. \

soc

This parameter is optional.

4. Customize the configuration file for your board as needed.

   For example, if your board is some customized version of the P2020rdb board, do the following:

   1. Copy the file **P1_P2_RDB.h** file and provide a new name (for example, **P1_P2_Custom.h**).
   2. Customize its content to match your custom hardware by editing the new file.
   3. In the file, **/home/wruser/u-boot/boards.cfg**, add the following configuration line:

   ```
   Active powerpc mpc85xx - - P1_P2_Custom P1_P2_CUSTOM P1_P2_Custom:P1_P2_CUSTOM
   -
   ```

Previous topic: Obtaining and Installing the U-Boot Source Tree on page 2
Next topic: Installing and Configuring a Toolchain to Build U-Boot on page 5

3. Installing and Configuring a Toolchain to Build U-Boot

You must obtain, install, and configure a toolchain in order to build U-Boot.

About This Task

The toolchain you obtain and configure must be suitable for your architecture. Obtaining and configuring a toolchain is beyond the scope of this document. The following example uses a GNU toolchain that is configured using a set of environment variables. Your toolchain may require a different configuration procedure.

Procedure

1. To inform the GNU compiler that you are building a PowerPC binary, execute the following command:

   ```
   $ export ARCH=powerpc
   ```

2. To set the **CROSS_COMPILE** environment variable, which specifies the prefix of GNU cross-compile tools, execute the following command:

```
$ export CROSS_COMPILE=powerpc-wrs-linux-gnu-
```

3. Add the path for the GNU toolchain to the standard search path by adding it to the **PATH** environment variable as follows:

```
$ export PATH=$PATH:/home/wruser/toolchain/wrll-toolchain-4.4a-341/powerpc/toolchain/x86-linux2
/bin
```

Previous topic: Updating the U-Boot Configuration Files on page 2
Next topic: Building the U-Boot Binary on page 6

4. Building the U-Boot Binary

You must create a U-Boot binary for the P2020rdb board. This binary is used to flash U-Boot onto the target board.

About This Task

The build process takes approximately two minutes to complete. When the build is finished, a **u-boot.bin** file is included under the **/home/wruser/u-boot** directory.

The U-Boot image is also available in **srec** format (**u-boot.srec**) so that it can be used with a flash programmer.

Procedure

1. Navigate to the main U-Boot install directory, **/home/wruser/u-boot/**.
2. To clean the work directory, execute the following command:

```
$ make distclean
```

3. To select the P2020rdb target board, execute the following command:

```
$ make P2020RDB_config
```

4. To build the U-Boot binary, execute the make command.

```
$ make
```

Previous topic: Installing and Configuring a Toolchain to Build U-Boot on page 5
Next topic: Transferring the U-Boot Binary to your Target on page 6

5. Transferring the U-Boot Binary to your Target

You must transfer the U-Boot binary to your target. A common method is to program the U-Boot executable into flash.

About This Task

You must transfer the U-Boot executable to your boot media. Boot media can include flash, an SD card, or another type of media, depending on your hardware. For this tutorial, flash is used.

Procedure

1. Program the U-Boot binary to flash address 0xeff80000 which is the default boot address for the P2020rdb board.

   Depending on your flash programmer, you may need to use the **u-boot.srec** f ile (also under **/home/wruser/u-boot**).

2. Power up your board.

Results

The U-Boot prompt appears as follows: =>.

```
U-Boot 2014.01-rc1 (Feb 18 2015 - 13:51:01)

CPU0:  P2020, Version: 2.0, (0x80e20020)
Core:  e500, Version: 4.0, (0x80211040)
Clock Configuration:
       CPU0:1200 MHz, CPU1:1200 MHz,
       CCB:800  MHz,
       DDR:200  MHz (400 MT/s data rate) (Asynchronous), LBC:200  MHz
L1:    D-cache 32 KiB enabled
       I-cache 32 KiB enabled
Board: P2020RDB RevC
I2C:   ready
SPI:   ready
DRAM:  Configuring DDR for 400 MT/s data rate
1 GiB (DDR2, 64-bit, CL=3, ECC off)

Flash: 16 MiB
L2:    512 KiB enabled
NAND:  1024 MiB
MMC:  voltage not supported by controlle
r
PCIe1: Root Complex of Slot 2, no link, regs @ 0xffe0a000
PCIe1: Bus 00 - 00
PCIe2: disabled
In:    serial
Out:   serial
Err:   serial
Net:   uploading VSC7385 microcode from ef000000
Failure uploading VSC7385 microcode.
eTSEC1 [PRIME], eTSEC2, eTSEC3
Hit any key to stop autoboot:  0
=>
```

Previous topic: Building the U-Boot Binary on page 6
Next topic: Booting your VxWorks Image from Flash on page 7

6. Booting your VxWorks Image from Flash

Once you have installed and configured U-Boot, you can boot your VxWorks image from flash.

About This Task

This tutorial boots the VxWorks image from flash. However, you can boot your VxWorks image from other media types such as an SD card or a TFTP server. (For information on booting your image from other media types, see the VxWorks 7 Boot Loader User's Guide.)

U-Boot boots a VxWorks kernel image in a **uVxWorks** format which is a binary format (**VxWorks.bin**) with a small U-Boot header attached.

Procedure

1. Create the **uVxWorks** image as follows:
   1. In Workbench, select your VxWorks image project (VIP).
   2. Expand **Build Targets**.
   3. Select the **uVxWorks** image. Right-click and select **Build Target**.

   The **uVxWorks** image and the DTB files are now available under **your_VIP_dir/default**.

2. Use the bootm command to boot your VxWorks 7 image from flash.

   The general command syntax is as follows:

   ```
   bootm kernel_addr - dtb_addr
   ```

   ```
   => bootm 0xef000000 - 0xef400000
   ```

   The **uVxWorks** image is programmed into flash at address 0xef000000. The device tree binary (DTB) file is programmed to 0xef400000.

   ```
   ## Booting kernel from Legacy Image at ef000000 ...
      Image Name:   vxworks
      Created:      2015-01-14  17:06:52 UTC
      Image Type:   PowerPC VxWorks Kernel Image (gzip compressed)
      Data Size:    962330 Bytes = 939.8 KiB
      Load Address: 00100000
      Entry Point:  00100000
      Verifying Checksum ... OK
   ## Flattened Device Tree blob at ef400000
      Booting using the fdt blob at 0xef400000
      Uncompressing Kernel Image ... OK
      Loading Device Tree to 03ffa000, end 03fff6c3 ... OK
   ## Starting vxWorks at 0x00100000, device tree at 0x03ffa000 ...
   Hello, VxWorks!
   check system clock frequency
   …
   …

    ________            ________
    \77777777\          /77777777/
     \77777777\        /77777777/
      \77777777\      /77777777/
       \77777777\    /77777777/
        \77777777\   \777777/
         \77777777\   \77777/              VxWorks 7
          \77777777\   \777/
   ```

```
        \77777777\   \7/     Core Kernel version: 1.0.6.0
         \77777777\   -      Build date: Jan 14 2015 09:47:15
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                 1984-2014
             \777/   /7\
              \7/   /777\
               -   -------

             Board: Freescale P2020RDB
     OS Memory Size: 1024MB
   ED&R Policy Mode: Deployed


 Adding 8190 symbols for standalone.

->
```

Previous topic: Transferring the U-Boot Binary to your Target on page 6
Next topic: Providing Default Environment Variables for U-Boot on page 9

7. Providing Default Environment Variables for U-Boot

You can create and run a script to set the default U-Boot environment variables.

About This Task

There are three main objectives for using environment variables in U-Boot:

- To provide parameters for U-Boot such as the target IP address, the host IP address, and so forth.
- To use U-Boot environment variables to overwrite the device tree binary (DTB). (One use is to modify the VxWorks 7 bootline without having to modify the DTB file. However, any content in the DTB can be overwritten.)
- To create new U-Boot environment variables that can be used as custom U-Boot commands.

Note: U-Boot environment variables can be hardcoded into U-Boot by adding them to your target configuration file and then rebuilding U-Boot. However, the most convenient way is to execute a U-Boot script.

Procedure

1. In an editor, create a script named **setenv-commands** that initializes a set of environment variables and saves them into flash.

   The script is as follows:

```
setenv ipaddr 10.10.0.100
setenv gatewayip 10.10.0.1
setenv serverip 10.10.0.240
setenv bootargs 'motetsec(0,0)host:vxWorks h=10.10.0.240 e=10.10.0.100:fffffe00 g=10.10.0.1 u=w
ruser pw=wruser f=0x80'
setenv kernel images/uVxWorks
setenv dtb images/p2020rdb.dtb
```

```
setenv load_kernel 'tftp 0x3000000 ${kernel};'
setenv load_dtb 'tftp 0xf000000 ${dtb};'
setenv bootm_cmd 'bootm 0x3000000 - 0xf000000;'
setenv vxboot run load_kernel load_dtb bootm_cmd
saveenv
```

2. In a VxWorks 7 development shell, use the mkimage command to convert the text script into an image.

```
> mkimage -T script -C none -n 'Demo u-boot script' -d setenv-commands setenv.img
```

The following output appears:

```
Image Name:   Demo u-boot script
Created:      Mon Feb 23 13:37:49 2015
Image Type:   PowerPC Linux Script (uncompressed)
Data Size:    467 Bytes = 0.46 kB = 0.00 MB
Load Address: 00000000
Entry Point:  00000000
Contents:
   Image 0: 459 Bytes = 0.45 kB = 0.00 MB
```

3. Copy the file **setenv.img** to **/tftpboot/images**.
4. From the U-boot shell, load the script.

```
=> tftp 0x2000000 images/setenv.img
Speed: 1000, full duplex
Using eTSEC2 device
TFTP from server 10.10.0.240; our IP address is 10.10.0.100
Filename 'images/setenv.img'.
Load address: 0x2000000
Loading: #
         0 Bytes/s
done
Bytes transferred = 531 (213 hex)
=>
```

5. Examine the status of the script image using the imi command.

```
=> imi 0x2000000

## Checking Image at 02000000 ...
   Legacy image found
   Image Name:   Demo u-boot script
   Created:      2015-02-23  18:37:49 UTC
   Image Type:   PowerPC Linux Script (uncompressed)
   Data Size:    467 Bytes = 467 Bytes
   Load Address: 00000000
   Entry Point:  00000000
   Contents:
      Image 0: 459 Bytes = 459 Bytes
   Verifying Checksum ... OK
=>
```

6. Run the script using the source command.

```
=> source 0x2000000
## Executing script at 02000000
Saving Environment to Flash...
Un-Protected 1 sectors
Erasing Flash...
. done
Erased 1 sectors
Writing to Flash... 9....8....7....6....5....4....3....2....1....9....8....7....
6....5....4....3....2....1....done
Protected 1 sectors
=>
```

Note: All environment variables are now stored in flash for future reboots.

7. Load the kernel, the DTB, and start VxWorks using the run command.

```
=> run vxboot
Speed: 1000, full duplex
Using eTSEC2 device
TFTP from server 10.10.0.240; our IP address is 10.10.0.100
Filename 'images/uVxWorks'.
Load address: 0x3000000
Loading: ##################################################################
         ##################################################################
         ########################
         548.8 KiB/s
done
Bytes transferred = 2245184 (224240 hex)
Speed: 1000, full duplex
Using eTSEC2 device
TFTP from server 10.10.0.240; our IP address is 10.10.0.100
Filename 'images/p2020rdb.dtb'.
Load address: 0xf000000
Loading: #
         3.9 KiB/s
done
Bytes transferred = 9924 (26c4 hex)
WARNING: adjusting available memory to 30000000
## Booting kernel from Legacy Image at 03000000 ...
   Image Name:   vxworks
   Created:      2015-02-18  15:47:06 UTC
   Image Type:   PowerPC VxWorks Kernel Image (uncompressed)
   Data Size:    2245120 Bytes = 2.1 MiB
   Load Address: 00100000
   Entry Point:  00100000
   Verifying Checksum ... OK
## Flattened Device Tree blob at 0f000000
   Booting using the fdt blob at 0xf000000
   Loading Kernel Image ... OK
   Loading Device Tree to 03ffa000, end 03fff6c3 ... OK
## Starting vxWorks at 0x00100000, device tree at 0x03ffa000 ...
Hello, VxWorks!
…
…
      \77777777\   \77777/              VxWorks 7
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 1.0.6.0
```

```
         \77777777\   -      Build date: Feb 18 2015 10:47:05
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                 1984-2014
             \777/   /7\
              \7/   /777\
               -   -------

              Board: Freescale P2020RDB
     OS Memory Size: 1024MB
   ED&R Policy Mode: Deployed


 Adding 8190 symbols for standalone.

-> ifconfig
…
…
motetsec0       Link type:Ethernet  HWaddr 00:a0:1e:a0:a1:a2
        capabilities: TXCSUM TX6CSUM VLAN_MTU
        inet 10.10.0.100  mask 255.255.254.0  broadcast 10.10.1.255
        UP RUNNING SIMPLEX BROADCAST MULTICAST
        MTU:1500  metric:1  VR:0  ifindex:2
        RX packets:5 mcast:0 errors:0 dropped:0
        TX packets:29 mcast:0 errors:0
        collisions:0 unsupported proto:0
        RX bytes:300  TX bytes:1434

value = 0 = 0x0
->
```

Note that the IP address for the **motetsec0** interface is now 10.10.0.100 as configured by the bootargs environment variable in the script.

> **Note:** Because the environment variables are saved in flash, after a reboot, all you need to issue at the U-Boot shell is the run vxboot command.

Previous topic: Booting your VxWorks Image from Flash on page 7